

ui.sub_pages 全面详解

`ui.sub_pages` 是 NiceGUI 框架中用于实现**基于 URL 的视图导航**的核心组件，专门用于构建单页应用（SPA）。它作为当前活跃子页面的容器，能在 URL 变化时无刷新切换视图，无需手动处理页面重载逻辑，同时支持数据传递、异步加载、嵌套路由等丰富功能，是构建复杂页面结构的关键工具。

核心特性与基础用法

1. 核心作用

- 实现 URL 与视图的映射，支持 `/`、`/other` 等路由格式；
- 切换路由时仅替换容器内内容，不触发全页刷新，保持 SPA 流畅体验；
- 自动管理路由匹配与视图渲染，简化导航逻辑。

2. 基础示例（最小可用代码）

```
from nicegui import ui

def root():
    # 定义路由映射：路径 -> 视图构造函数
    ui.sub_pages({
        '/': main_page,    # 根路径对应 main_page 函数
        '/other': other_page # /other 路径对应 other_page 函数
    })

def main_page():
    ui.label('主页面内容')
    ui.link('前往其他页面', '/other') # 路由跳转链接

def other_page():
    ui.label('其他页面内容')
    ui.link('返回主页面', '/')

ui.run(root)
```

- 运行后访问根路径 `/` 显示主页面，点击链接可切换至 `/other` 路径，URL 同步更新；
- 页面 ID 仅在手动刷新时变化，证明无全页重载。

关键功能详解

1. 向子页面传递数据

通过 `ui.sub_pages` 的 `data` 参数传递字典格式数据，子页面函数可通过**关键字参数**或 `PageArguments.data` 接收数据，适用于父子页面数据共享（如状态同步、组件传递）。

示例：传递标签组件实现标题同步

```
from nicegui import ui
```

```

def root():
    with ui.row():
        ui.label('当前标题: ')
        title_label = ui.label() # 待传递给子页面的组件

    ui.sub_pages({
        '/': main_page,
        '/other': other_page,
    }, data={'title': title_label}) # 传入数据字典

# 子页面通过关键字参数接收数据（类型提示可选但推荐）
def main_page(title: ui.label):
    title.text = '主页面' # 修改父页面传递的组件状态
    ui.button('前往其他页面', on_click=lambda: ui.navigate.to('/other'))

def other_page(title: ui.label):
    title.text = '其他页面'
    ui.button('返回主页面', on_click=lambda: ui.navigate.to('/'))

ui.run(root)

```

2. 异步子页面

支持异步视图构造函数（`async def` 定义），可处理异步操作（如接口请求、延迟加载），视图渲染过程不阻塞主线程。

示例：带延迟的异步页面

```

import asyncio
from nicegui import ui

def root():
    # 导航链接
    with ui.row():
        ui.link('主页面', '/')
        ui.link('其他页面', '/other')

    ui.sub_pages({
        '/': main_page,
        '/other': lambda: other_page('异步页面') # 传递参数给异步函数
    })

async def main_page():
    ui.label('主页面（异步）').classes('font-bold')
    await asyncio.sleep(2) # 模拟异步操作（如接口请求）
    ui.label('2秒后加载的内容')

async def other_page(title: str):
    ui.label(title).classes('font-bold')
    await asyncio.sleep(1)
    ui.label('1秒后加载的内容')

```

```
ui.run(root)
```

3. 动态添加路由

若初始化时无法确定所有路由（如根据配置动态生成页面），可先创建空的 `ui.sub_pages` 实例，再通过 `add()` 方法动态添加路由。

示例：动态路由注册

```
from nicegui import ui

def root():
    # 创建空的 sub_pages 容器
    pages = ui.sub_pages()
    ui.separator()
    footer = ui.label('默认页脚') # 共享页脚组件

    # 动态添加路由（路径，视图函数）
    pages.add('/', lambda: main_page(footer))
    pages.add('/other', lambda: other_page(footer))

def main_page(footer: ui.label):
    footer.text = '主页面页脚'
    ui.link('前往其他页面', '/other')

def other_page(footer: ui.label):
    footer.text = '其他页面页脚'
    ui.link('返回主页面', '/')

ui.run(root)
```

4. URL 参数传递与解析

支持**路径参数**（`/item/{item_id}`）和**查询参数**（`?color=red`），并自动根据类型提示转换参数类型（如 `int`、`float`），未指定默认值的参数为必填项。

示例：带参数的路由

```
from nicegui import ui

def root():
    ui.sub_pages({
        '/': main_page,
        '/item/{item_id}': item_page # 路径参数: item_id
    })

def main_page():
    ui.link('查看物品1', '/item/1')
    ui.link('查看物品2', '/item/2')
    ui.link('查看红色物品3', '/item/3?color=red') # 路径+查询参数

# 子页面接收参数（item_id 为必填路径参数，color 为可选查询参数，默认值 blue）
```

```
def item_page(item_id: int, color: str = 'blue'):
    ui.label(f'物品 ID: {item_id}').classes(f'font-bold text-2xl text-{color}')
    ui.link('返回', '/')

ui.run(root)
```

- 路径参数 `item_id` 自动转换为 `int` 类型;
- 查询参数 `color` 若未传递, 使用默认值 `blue`。

5. 使用 PageArguments 统一访问参数

通过 `PageArguments` 类型提示, 可在子页面中统一获取**查询参数**、**路径参数**、**数据**等信息, 适用于参数较多或动态参数场景。

示例: `PageArguments` 用法

```
from nicegui import PageArguments, ui

def root():
    ui.link('传递消息: hello', '/?msg=hello')
    ui.link('传递消息: world', '/?msg=world')
    ui.sub_pages({'/' : main_page})

def main_page(args: PageArguments):
    # 从查询参数中获取 msg, 无则显示默认值
    msg = args.query_parameters.get('msg', '无消息')
    ui.label(f'收到消息: {msg}')
    # 额外: 访问路径参数 (若有) args.path_parameters, 或父页面传递的数据 args.data

ui.run(root)
```

`PageArguments` 核心属性:

- `query_parameters`: 字典类型, 存储所有查询参数;
- `path_parameters`: 字典类型, 存储所有路径参数;
- `data`: 父页面通过 `ui.sub_pages(data=...)` 传递的数据。

6. 嵌套子页面 (层级路由)

支持 `ui.sub_pages` 嵌套, 实现层级化路由结构 (如 `/other/a`、`/other/b`), 每个嵌套的 `ui.sub_pages` 仅处理 URL 中未被父级处理的部分。

示例: 二级嵌套路由

```
from nicegui import ui

def root():
    # 一级路由
    ui.link('主页面', '/')
    ui.link('其他页面', '/other')
    ui.sub_pages({
```

```
        '/': main_page,
        '/other': other_page
    }).classes('border p-2') # 样式: 边框+内边距

def main_page():
    ui.label('一级主页面')

def other_page():
    # 二级路由 (处理 /other 后的路径部分)
    ui.label('一级其他页面')
    ui.link('前往二级页面 A', '/other/a')
    ui.link('前往二级页面 B', '/other/b')

    # 嵌套的二级 sub_pages
    ui.sub_pages({
        '/': sub_main_page, # 匹配 /other (二级根路径)
        '/a': sub_page_a,   # 匹配 /other/a
        '/b': sub_page_b    # 匹配 /other/b
    }).classes('border p-2 ml-4') # 样式: 缩进+边框

def sub_main_page():
    ui.label('二级主页面 (/other)')

def sub_page_a():
    ui.label('二级页面 A (/other/a)')

def sub_page_b():
    ui.label('二级页面 B (/other/b)')

ui.run(root)
```

嵌套路由匹配规则:

- 1. 父级 `ui.sub_pages` 处理 URL 前缀部分 (如 `/other`) ;
- 2. 子级 `ui.sub_pages` 处理剩余路径 (如 `a`、`b`) ;
- 3. 若路径无匹配项, 默认显示 404 错误 (可通过 `show_404` 参数控制) 。

完整 API 参考

1. 初始化参数 (Initializer)

参数名	类型	说明
routes	dict[str, Callable]	路由映射字典: 键为路径模式 (如 <code>/</code> 、 <code>/item/{id}</code>) , 值为视图构建函数
root_path	str	路径前缀 (嵌套 <code>ui.sub_pages</code> 时忽略) , 用于统一剥离路径前缀
data	dict[str, Any]	传递给所有子页面的数据, 可通过关键字参数或 <code>PageArguments.data</code> 访问

参数名	类型	说明
show_404	bool	路径无匹配时是否显示 404 错误（默认 True）

2. 核心属性 (Properties)

属性名	类型	说明
classes	Classes[Self]	组件样式类（支持 Tailwind、Quasar 类）
client	Client	组件所属的客户端实例
html_id	str	HTML DOM 中的元素 ID（v2.16.0+）
visible	BindableProperty	组件可见性（可绑定到其他对象的属性）
is_deleted	bool	组件是否已被删除

3. 关键方法 (Methods)

方法名	参数说明	返回值	说明
add(path, page)	path: 路径模式; page: 视图构造函数	Self	动态添加路由（支持链式调用，如 <code>pages.add('/a', a).add('/b', b)</code> ）
refresh()	无	None	重建子页面组件（v3.1.0+），用于强制刷新视图
clear()	无	None	删除所有子元素（清空当前视图）
delete()	无	None	删除组件及其所有子元素
bind_visibility(...)	target_object: 绑定对象; target_name: 属性名; forward/backward: 转换函数	Self	双向绑定组件可见性
tooltip(text)	text: 提示文本	Self	为组件添加悬停提示

4. 继承关系

`ui.sub_pages` 继承自 NiceGUI 的核心基类，具备基础组件的所有通用能力：

- `Element`：所有 UI 组件的基类，提供 `classes`、`props`、`style` 等样式控制；
- `visibility`：提供可见性控制相关方法（如 `set_visibility`、`bind_visibility`）。

最佳实践与注意事项

- 路由命名规范**：路径建议以 `/` 开头（如 `/other` 而非 `other`），避免匹配异常；
- 参数类型提示**：路径 / 查询参数建议添加类型提示（如 `item_id: int`），NiceGUI 会自动转换类型，未指定

则为 `str`;

3. **嵌套路由层级**：嵌套层级无强制限制，但建议控制在 2-3 层内，避免 URL 过长和路由逻辑复杂；
4. **404 自定义**：若需自定义 404 页面，可添加路径 `'*'` 映射到自定义视图函数（如 `'*': not_found_page`）；
5. **动态路由场景**：适合权限控制后的路由注册（如登录后添加用户专属路由）、配置驱动的面面生成等场景。

总结

`ui.sub_pages` 是 NiceGUI 构建 SPA 的核心组件，通过 URL 路由映射实现无刷新视图切换，支持数据传递、异步加载、嵌套路由、参数解析等全方位功能。其 API 设计简洁直观，既满足简单页面的基础导航需求，也能支撑复杂层级的企业级应用开发，是 NiceGUI 中不可或缺的关键工具。

NiceGUI 中 `ui.sub_pages` 的全维度解析

`ui.sub_pages` 是 NiceGUI 用于构建**多页面应用（MPA）**的核心组件，支持在单个应用中创建多个独立的子页面（路由），实现页面间的导航、参数传递、权限控制等功能。它是替代 `ui.page` 手动管理路由的高阶封装，大幅简化多页面应用的开发流程。

一、核心作用与适用场景

1. 核心作用

- 基于 URL 路由自动匹配并渲染对应子页面（如 `/home` → 首页、`/settings` → 设置页）；
- 统一管理所有子页面的导航、布局（如共享顶部导航栏、侧边栏）；
- 支持页面参数传递（如 `/user/123` → 获取用户 ID=123）、嵌套子页面；
- 与 NiceGUI 的状态管理、认证鉴权无缝集成。

2. 典型适用场景

场景	示例
后台管理系统	首页、用户管理、订单管理、设置等子页面
多模块 Web 应用	数据看板、报表、配置、帮助等子页面
带参数的动态页面	<code>/product/1</code> 、 <code>/product/2</code> 等商品详情页
嵌套路由页面	<code>/dashboard/stats</code> 、 <code>/dashboard/settings</code> 等嵌套子页面

二、基本语法与使用方式

1. 核心概念

- **路由 (Route)**：子页面的 URL 路径（如 `/home`、`/user/{id}`）；
- **布局 (Layout)**：所有子页面共享的通用 UI（如导航栏、侧边栏）；
- **页面函数**：每个子页面对应的渲染函数，返回该页面的 UI 内容；
- **参数提取**：从路由中提取动态参数（如 `/user/{id}` 中的 `id`）。

2. 基础语法

```
from nicegui import ui

# 1. 定义共享布局（可选，推荐）
def main_layout():
    # 所有子页面共享的顶部导航
    with ui.page_sticky(position='top', offset=0).classes('bg-blue-500 text-white p-2 w-full'):
        with ui.row():
            ui.page_scroller(target='/home').text('首页') # 跳转到首页
            ui.page_scroller(target='/settings').text('设置') # 跳转到设置页
            ui.space()
            ui.label('多页面应用示例')
    # 布局的内容区域（子页面会渲染在这里）
    ui.separator()

# 2. 定义子页面函数
def home_page():
    ui.label('首页内容').classes('text-3xl text-center my-5')
    ui.button('前往设置页', on_click=lambda: ui.open('/settings'))

def settings_page():
    ui.label('设置页内容').classes('text-3xl text-center my-5')
    ui.button('返回首页', on_click=lambda: ui.open('/home'))

# 3. 初始化子页面管理器
sub_pages = ui.sub_pages(
    path='/', # 根路径（所有子页面的父路径）
    layout=main_layout, # 共享布局
    initial='/home' # 初始加载的子页面
)

# 4. 注册子页面
sub_pages.add('/home', home_page) # 注册首页
sub_pages.add('/settings', settings_page) # 注册设置页

ui.run()
```


3. 核心参数详解 (ui.sub_pages 初始化)

参数名	类型	取值说明	默认值
path	str	子页面的根路径 (如 /app, 则子页面路由为 /app/home、/app/settings)	'/'
layout	callable	共享布局函数: 无参数, 返回 / 渲染通用 UI (如导航栏), 子页面内容会渲染在布局之后	None
initial	str	应用启动时默认加载的子页面路由 (需已注册)	None
on_change	callable	页面切换时的回调函数, 接收参数 (old_path, new_path)	None

4. 动态参数路由 (核心进阶)

支持在路由中定义动态参数 (如 /user/{id}) , 并在页面函数中提取:

```
from nicegui import ui

# 定义带参数的页面函数 (接收路由参数)
def user_detail_page(id: str):
    ui.label(f'用户详情页 - ID: {id}').classes('text-2xl my-5')
    ui.button('返回用户列表', on_click=lambda: ui.open('/users'))

def user_list_page():
    ui.label('用户列表').classes('text-2xl my-5')
    # 生成带参数的跳转按钮
    for user_id in ['101', '102', '103']:
        ui.button(f'查看用户{user_id}', on_click=lambda id=user_id:
    ui.open(f'/user/{id}')).classes('mr-2')

# 初始化子页面
sub_pages = ui.sub_pages(initial='/users')
sub_pages.add('/users', user_list_page)
# 注册带动态参数的子页面 ({id}为参数占位符)
sub_pages.add('/user/{id}', user_detail_page)

ui.run()
```

5. 嵌套子页面

支持多层嵌套的子页面 (如 /dashboard/stats、/dashboard/settings) :

```
from nicegui import ui

# 父布局 (仪表盘通用布局)
def dashboard_layout():
    ui.label('仪表盘通用导航').classes('text-xl p-2 bg-gray-100')
    ui.separator()
```

```
# 子页面1: 统计页
def stats_page():
    ui.label('仪表盘 - 统计数据').classes('text-2xl my-5')

# 子页面2: 设置页
def dashboard_settings_page():
    ui.label('仪表盘 - 设置').classes('text-2xl my-5')

# 根布局
def root_layout():
    ui.page_scroller(target='/home').text('首页')
    ui.page_scroller(target='/dashboard').text('仪表盘')
    ui.separator()

# 根级子页面
root_sub_pages = ui.sub_pages(layout=root_layout, initial='/home')
root_sub_pages.add('/home', lambda: ui.label('首页')).classes('text-2xl')

# 嵌套子页面（仪表盘下的子页面）
dashboard_sub_pages = ui.sub_pages(path='/dashboard', layout=dashboard_layout)
dashboard_sub_pages.add('/stats', stats_page)
dashboard_sub_pages.add('/settings', dashboard_settings_page)

# 将嵌套子页面注册到根级
root_sub_pages.add('/dashboard', dashboard_sub_pages)

ui.run()
```

三、关键特性与注意事项

1. 页面切换方式

除了 `ui.page_scroller`，还可通过以下方式切换子页面：

- `ui.open(path)`：跳转到指定子页面（推荐）；
- `ui.navigate.to(path)`：底层导航 API，效果与 `ui.open` 一致；
- 直接修改 URL：在浏览器地址栏输入子页面路由（如 `http://localhost:8080/settings`）。

2. 布局与子页面的渲染顺序

- 若设置了 `layout`，渲染顺序为：布局 UI → 子页面 UI；
- 布局函数中可通过 `ui.column()` 等容器划分区域，子页面内容会追加到布局之后；
- 若需自定义子页面的渲染位置，可在布局中使用 `ui.container()` 并标记为 `slot`：

```
def custom_layout():
    with ui.row().classes('w-full'):
        # 侧边栏
        with ui.column().classes('w-1/4 bg-gray-100 p-2'):
            ui.label('侧边导航')
        # 子页面内容容器 (slot标记)
        with ui.column().classes('w-3/4 p-4').props('slot'):
            pass # 子页面内容会渲染到这里
```

3. 状态管理

子页面切换时，页面内的组件状态会重置（除非使用全局状态）：

```
from nicegui import ui
from nicegui.state import State

# 定义全局状态
class AppState(State):
    counter: int = 0

# 首页（修改全局状态）
def home_page():
    state = AppState()
    ui.label(f'计数器: {state.counter}').bind_text_from(state, 'counter')
    ui.button('+1', on_click=lambda: setattr(state, 'counter', state.counter + 1))
    ui.button('前往设置页', on_click=lambda: ui.open('/settings'))

# 设置页（读取全局状态）
def settings_page():
    state = AppState()
    ui.label(f'全局计数器值: {state.counter}').bind_text_from(state, 'counter')
    ui.button('返回首页', on_click=lambda: ui.open('/home'))

sub_pages = ui.sub_pages(initial='/home')
sub_pages.add('/home', home_page)
sub_pages.add('/settings', settings_page)

ui.run()
```

4. 页面切换回调

通过 `on_change` 参数监听页面切换事件：

```
from nicegui import ui

def on_page_change(old_path: str, new_path: str):
    ui.notify(f'从 {old_path} 切换到 {new_path}')

sub_pages = ui.sub_pages(
    initial='/home',
    on_change=on_page_change # 页面切换回调
)
sub_pages.add('/home', lambda: ui.label('首页'))
sub_pages.add('/settings', lambda: ui.label('设置页'))

ui.run()
```

5. 常见陷阱

- **路由冲突**: 避免注册重复的路由（如同时注册 `/user` 和 `/user/{id}`），需确保顺序，动态路由放后面）；
- **参数类型**: 动态路由参数默认是字符串，需手动转换为 `int/float` 等类型；
- **布局重复渲染**: 若 `layout` 函数中包含动态内容（如计数器），页面切换时会重新执行布局函数，导致状态重置（需用全局状态）；
- **初始路由失效**: `initial` 参数必须是已注册的路由，否则会显示空白页面。

6. 与 `ui.page` 的区别

`ui.sub_pages` 是多页面管理组件，而 `ui.page` 是单页面定义，二者核心差异：

特性	<code>ui.sub_pages</code>	<code>ui.page</code>
页面数量	多页面（路由管理）	单页面
共享布局	原生支持	需手动实现
动态参数	原生支持	需手动解析 URL
页面切换	路由跳转	需手动隐藏 / 显示组件

四、实战场景示例（后台管理系统）

```
from nicegui import ui

# 1. 定义全局状态
class AdminState(State):
    current_user: str = 'admin'

# 2. 定义主布局
def admin_layout():
    state = AdminState()
    # 顶部导航
```

```

with ui.page_sticky(position='top', offset=0).classes('bg-gray-800 text-white p-3 w-full'):
    with ui.row():
        ui.label('后台管理系统').classes('text-xl font-bold mr-8')
        ui.page_scroller(target='/dashboard').text('仪表盘').classes('mr-4')
        ui.page_scroller(target='/users').text('用户管理').classes('mr-4')
        ui.page_scroller(target='/orders').text('订单管理').classes('mr-4')
        ui.space()
        ui.label(f'当前用户: {state.current_user}').classes('text-sm')
ui.separator()

```

3. 定义子页面

```

def dashboard_page():
    ui.label('仪表盘 - 数据概览').classes('text-2xl my-5')
    with ui.row().classes('w-full'):
        ui.card(ui.label('今日订单: 120')).classes('w-1/3 p-4')
        ui.card(ui.label('今日用户: 35')).classes('w-1/3 p-4')
        ui.card(ui.label('销售额: ¥8900')).classes('w-1/3 p-4')

def users_page():
    ui.label('用户管理').classes('text-2xl my-5')
    # 模拟用户列表
    for i in range(5):
        with ui.row().classes('w-full p-2 border-b'):
            ui.label(f'用户{i+1}').classes('w-1/4')
            ui.label(f'user{i+1}@example.com').classes('w-2/4')
            ui.button('编辑', on_click=lambda id=i+1: ui.open(f'/user/{id}')).classes('w-1/4')

def user_detail_page(id: str):
    ui.label(f'编辑用户 - ID: {id}').classes('text-2xl my-5')
    ui.input('用户名').classes('w-1/2 my-2')
    ui.input('邮箱').classes('w-1/2 my-2')
    ui.button('保存', on_click=lambda: ui.notify('保存成功')).classes('mr-2')
    ui.button('返回', on_click=lambda: ui.open('/users'))

def orders_page():
    ui.label('订单管理').classes('text-2xl my-5')
    ui.button('导出订单', on_click=lambda: ui.notify('导出成功'))

```

4. 初始化并注册子页面

```

sub_pages = ui.sub_pages(layout=admin_layout, initial='/dashboard')
sub_pages.add('/dashboard', dashboard_page)
sub_pages.add('/users', users_page)
sub_pages.add('/user/{id}', user_detail_page)
sub_pages.add('/orders', orders_page)

ui.run(title='后台管理系统', port=8080)

```

